Указания за стартиране на платформата за публикуване и слушане на музика

Изисквания за стартиране на системата

За правилното стартиране и използване на разработената система е необходимо да бъдат инсталирани следните основни технологии и инструменти:

- **Visual Studio** с поддръжка на .NET разработка, използван за изграждане и изпълнение на сървърната част на приложението.
- **.NET 5**, който представлява основната платформа за разработка на backend логиката на системата.
- **Microsoft SQL Server**, използван като релационна база данни за съхранение на потребителски данни, музикално съдържание и свързаната с него информация.
- **Node.js (версия 14.21.3)**, необходим за стартиране и управление на клиентската част на приложението, разработена с Angular.

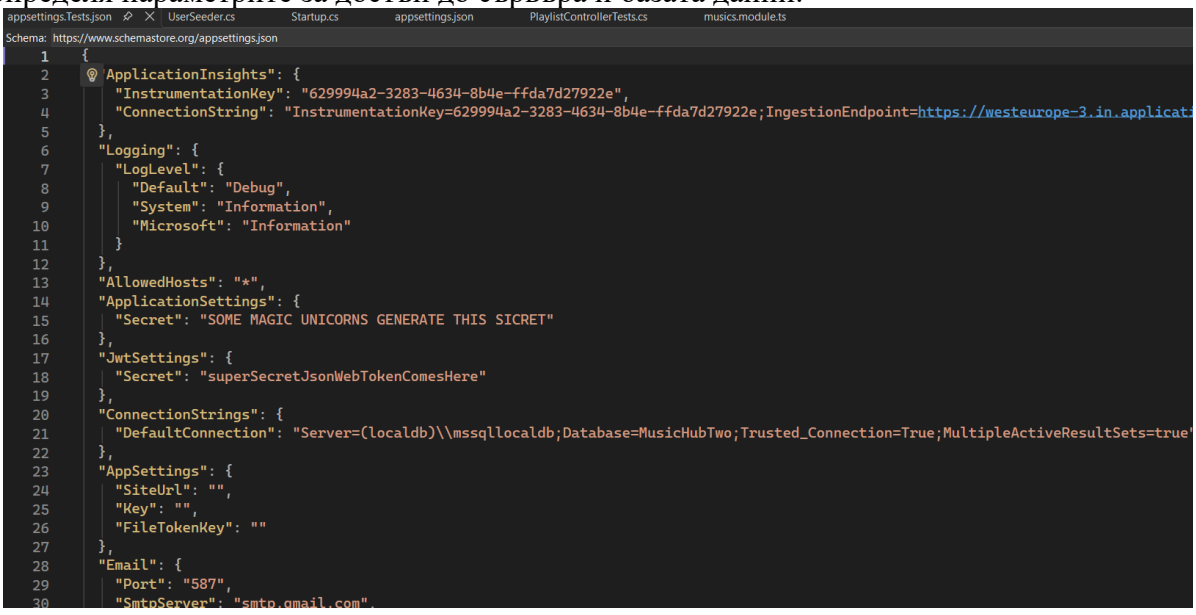
Посочените технологии осигуряват цялостната функционалност на системата, като позволяват изпълнение както на сървърната, така и на клиентската част на приложението в локална среда.

Линк към Github

Проектът може да бъде свален от https://github.com/dimitarkole/MusicHub

Конфигурация на базата данни

Връзката към базата данни е дефинирана в проекта MusicHub.Web във файла appsettings.json. В този файл се намира connection string-ът (DefaultConnection), който определя параметрите за достъп до сървъра и базата данни.



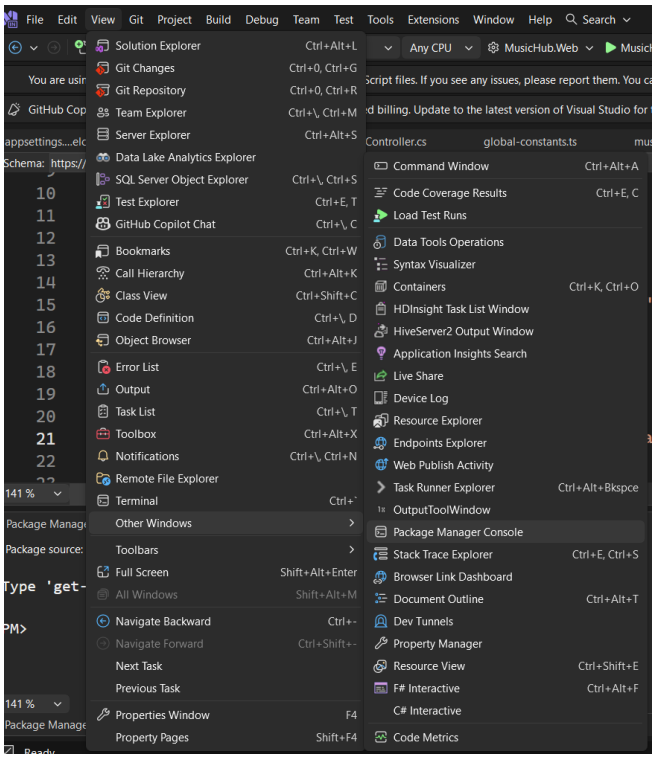
Фиг. 1 Съдържание на appsettings.json

При необходимост от локално стартиране на проекта, този параметър може да бъде променен според конкретната среда (локален SQL Server или друг сървър).

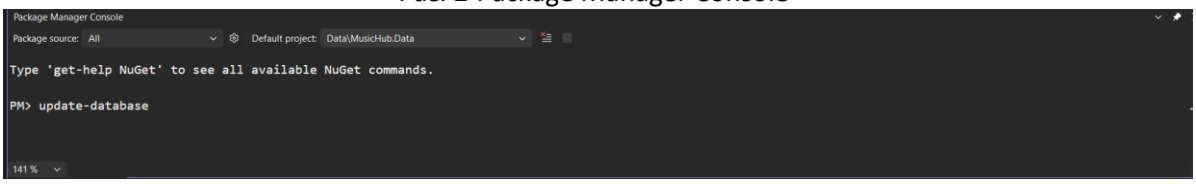
Създаване на база данни

За създаване и инициализиране на базата данни се използва подходът Entity Framework Core Migrations. След отваряне на проекта, в Package Manager Console се изпълнява следната команда (**Default project - MusicHub.Data**):

- `Update-Database`



Фиг. 2 Package Manager Console



Фиг. 3 Package Manager Console изпълняване на **update-database**

Тази команда създава необходимата структура в базата данни според дефинираните модели в приложението. Приложението може да бъде стартирано с F5.

Инициализация на тестови данни

В проекта е реализиран seeder, който се изпълнява при стартиране на приложението (в Startup.cs). Неговата цел е автоматично да зареди примерни данни в системата, които улесняват тестването и демонстрацията на функционалностите.

Seeder-ът създава предварително дефинирани потребители с различни роли и права, което позволява тестване на различни сценарии в системата.

Тестови потребители

За целите на демонстрация и тестване в системата са налични следните потребители:

Роля	Потребителско име	Парола
SuperAdministrator	rootadmin	rootpass
Administrator	rootadmin2	rootpass
DataAnalyst	rootadmin3	rootpass
User	rootuser	rootpass
User	rootuser1	rootpass
User	rootuser2	rootpass
User	pesho	rootpass
User	gosho	rootpass

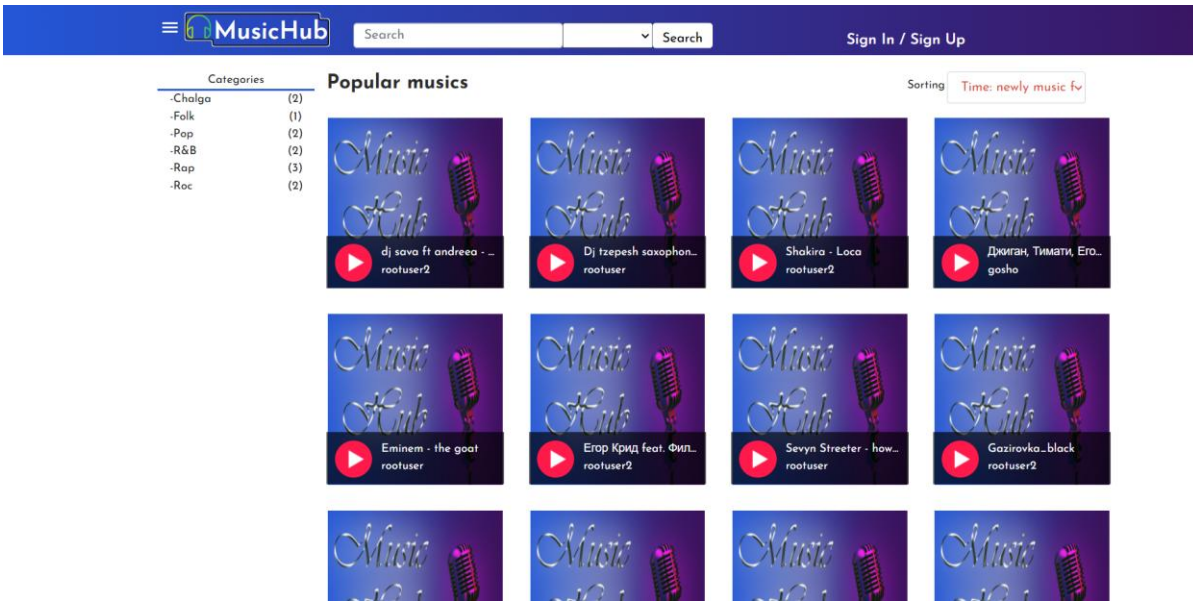
Таб. 6.1 Тестови потребители

Тези акаунти позволяват тестване на различните роли и функционалности в системата, включително административни и потребителски операции. За целите на тесването на двуфакторната верификация, моля, направете си регистрация в платформата.

Използване на приложението

- ### Начална страница

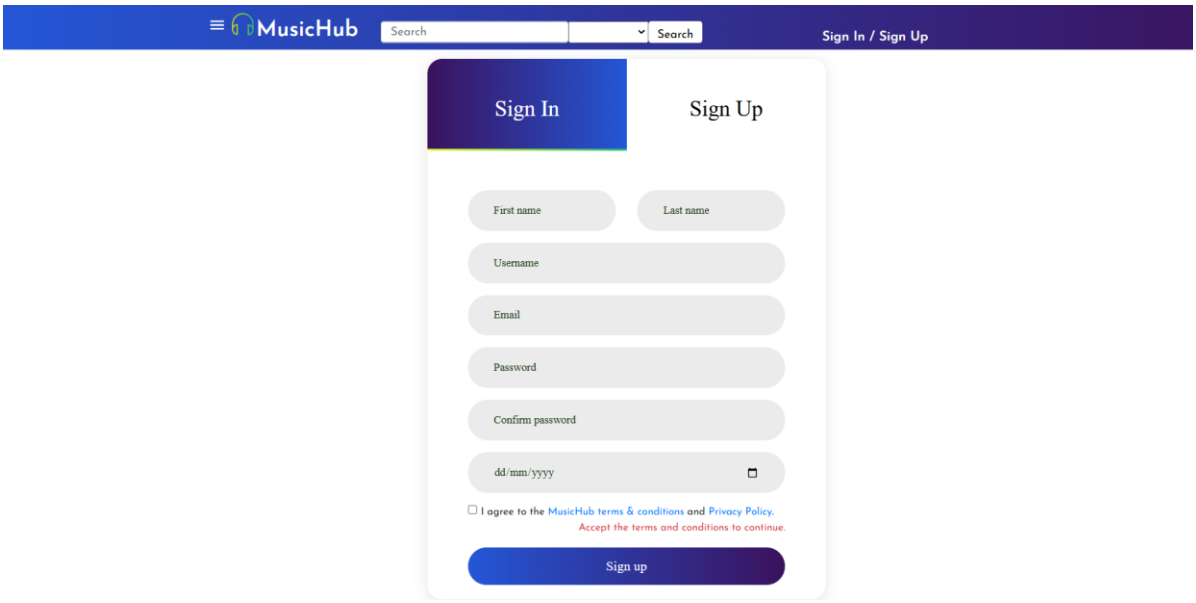
При отваряне на уеб приложението потребителят вижда началната страница, на която се визуализират всички публикувани музикални произведения. Зареждането на съдържанието е реализирано чрез механизъм за динамично извличане на данни (Infinite Scroll). При достигане до края на текущо заредените записи системата автоматично извлича следващата група песни от базата данни, без необходимост от презареждане на страницата. Това осигурява плавна навигация и по-добро потребителско изживяване.



Фиг. 4 Начален екран на приложението

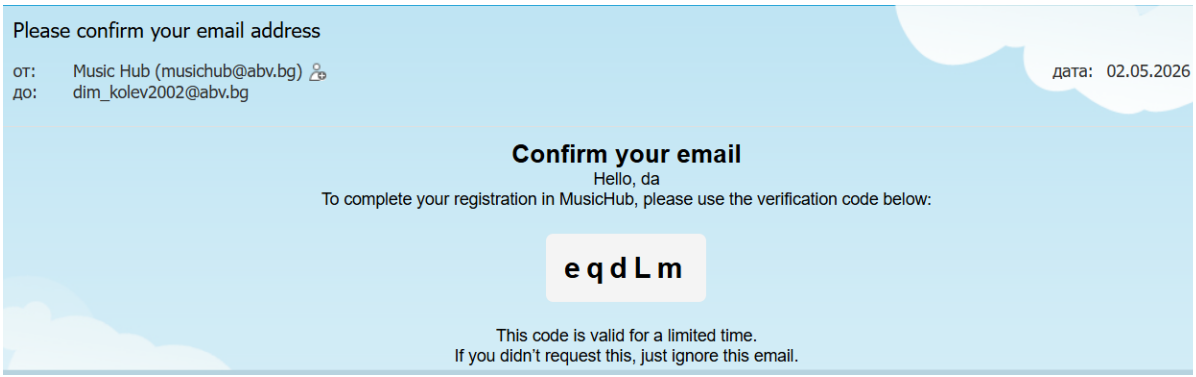
- **Регистрация на потребител**

За създаване на нов потребителски профил е необходимо да бъдат попълнени следните данни: име, фамилия, потребителско име, електронна поща, парола и дата на раждане. Преди завършване на регистрацията потребителят трябва да се съгласи с Общите условия на платформата, като маркира съответното поле. След попълване на всички задължителни данни регистрацията се извършва чрез натискане на бутона **Sign Up**.

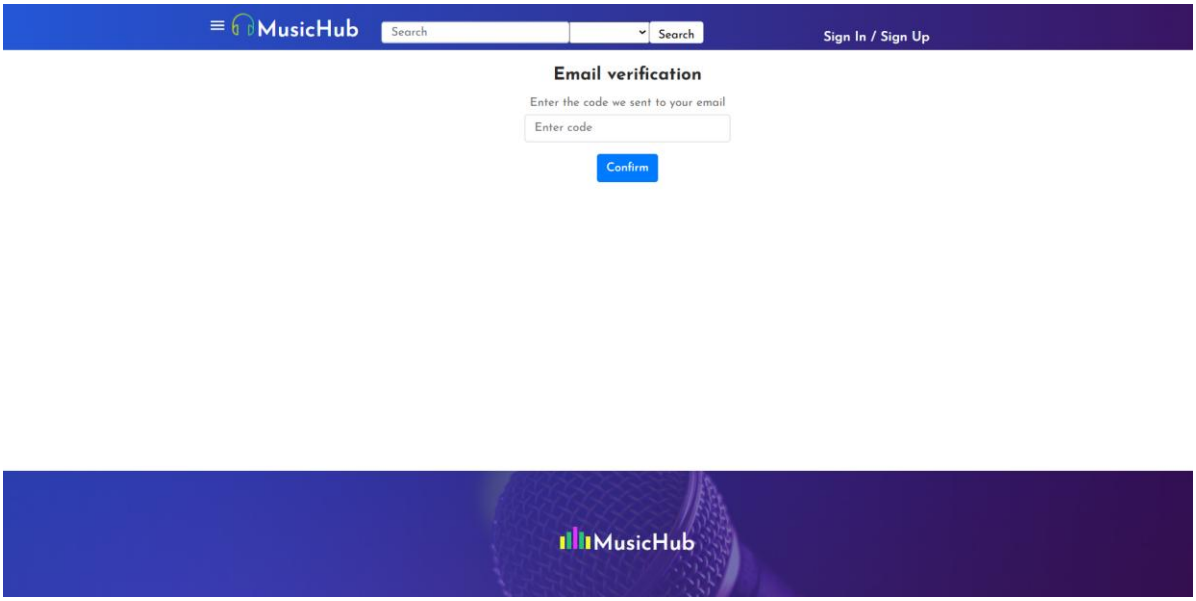


Фиг. 5 Форма за регистрация

След успешното създаване на профила системата изпраща съобщение за потвърждение на посочения електронен адрес. Потребителят трябва да отвори получения имейл и да потвърди своята регистрация чрез предоставената връзка, след което профилът се активира.



Фиг. 6 Съобщение за потвърждение на email

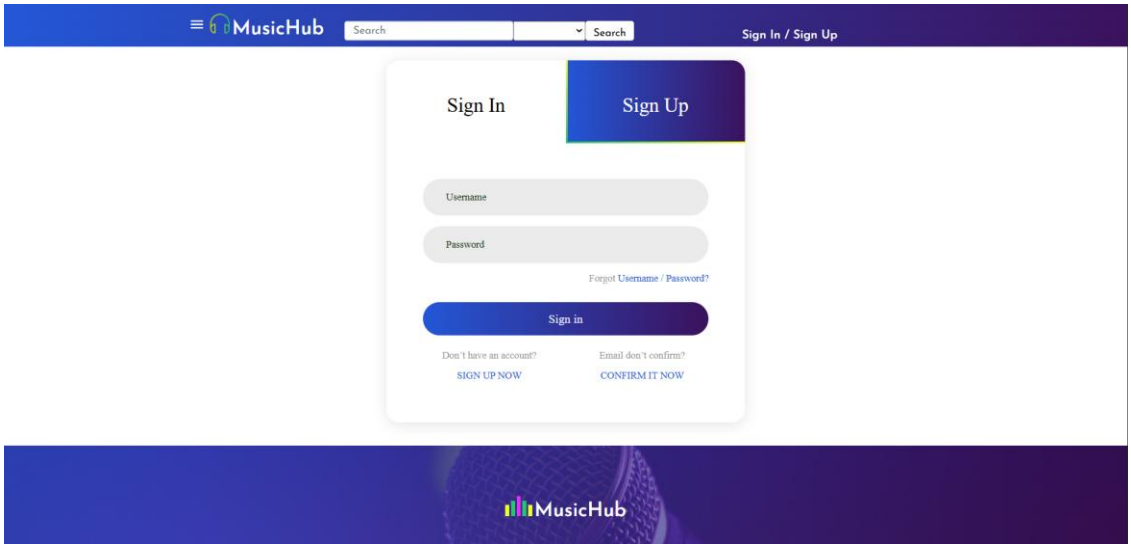


Фиг. 7 Въвеждане на код за потвърждение на email

За всеки новорегистриран потребител по подразбиране е активирана двуфакторна автентикация чрез електронна поща. При всяко влизане в системата на регистрирания имейл адрес се изпраща еднократен код за потвърждение, който трябва да бъде въведен, за да бъде завършен процесът на удостоверяване.

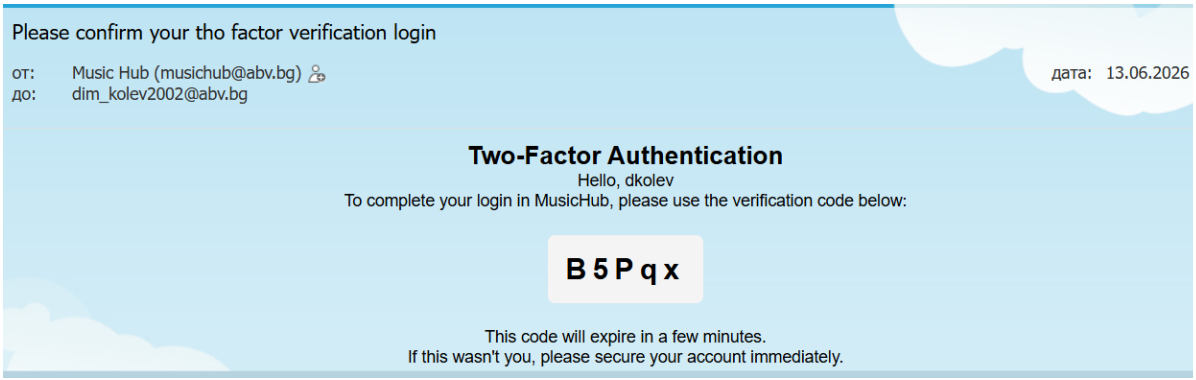
- **Вход в системата**

За достъп до платформата потребителят използва формата за вход, като трябва да въведе своето потребителско име и парола. След успешно валидиране на въведените данни системата стартира процес на двуфакторна автентикация.

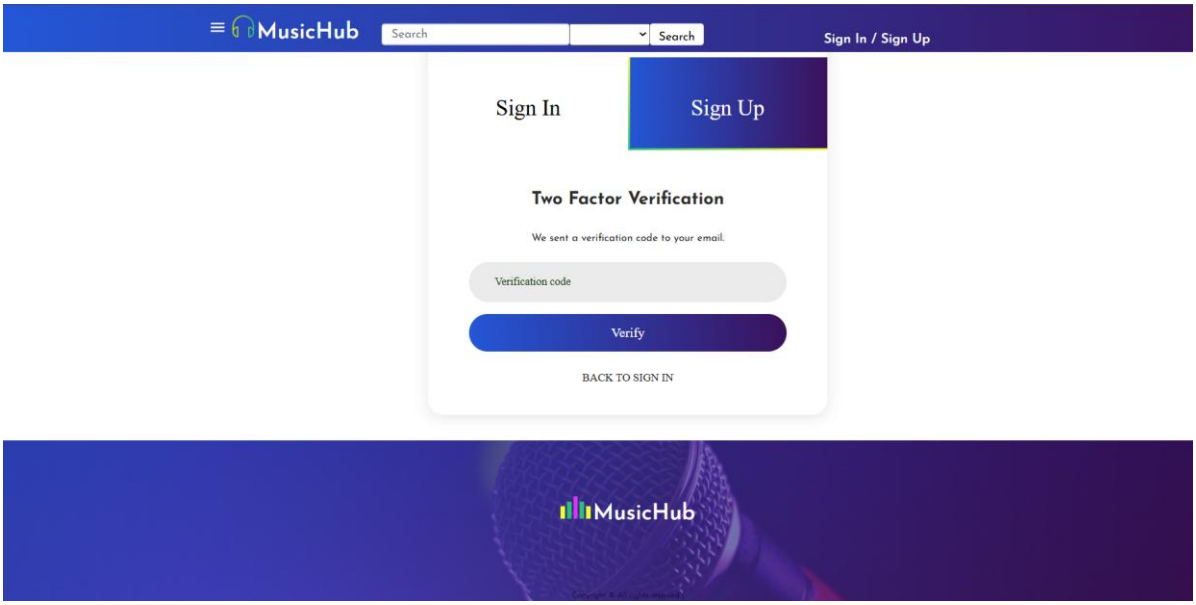


Фиг. 8 Форма за вход

В рамките на този процес на регистрирания електронен адрес на потребителя се изпраща еднократен код за потвърждение. Потребителят трябва да въведе получения код в системата, за да завърши успешно процеса на вход и да получи достъп до функционалностите на приложението.



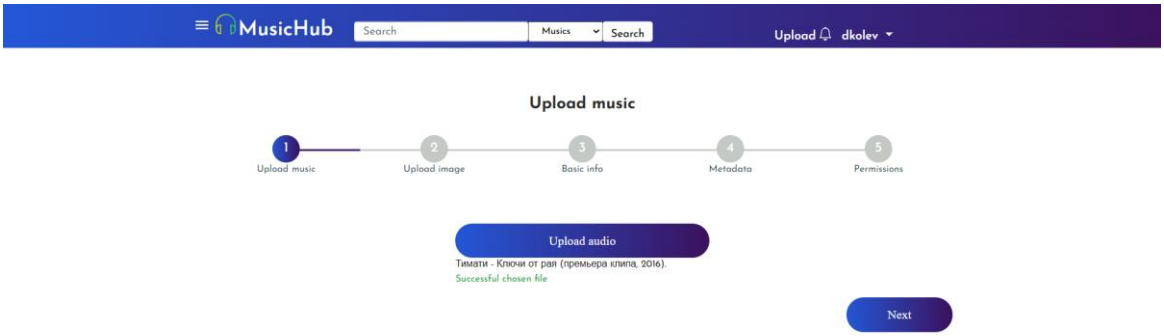
Фиг. 9 Two-Factor mail



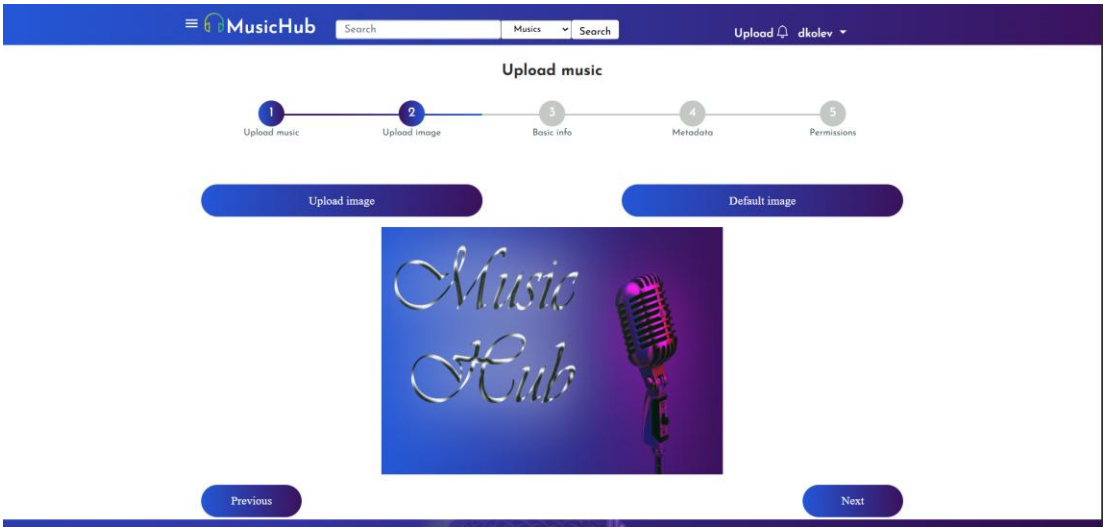
Фиг. 10 Въвеждане на код за Two-Factor mail

- **Добавяне на песен**

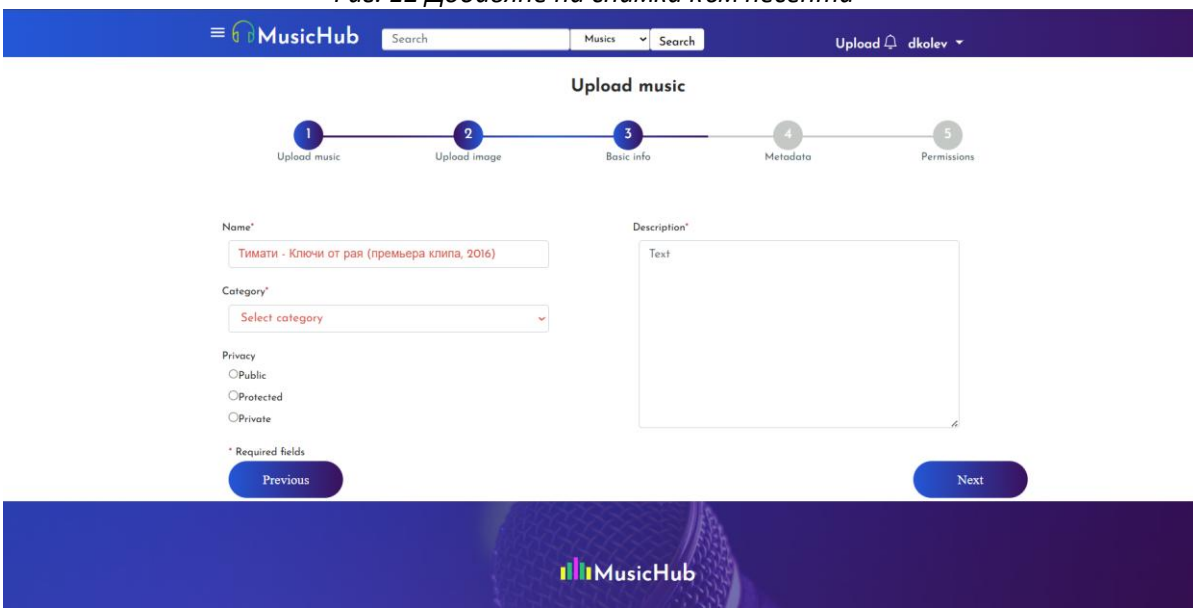
Процесът по добавяне на ново музикално съдържание в платформата е разделен на няколко последователни етапа с цел по-добра организация и валидиране на данните. Стъпките включват: **Upload music**, **Upload image**, **Basic info**, **Metadata** и **Permissions**.



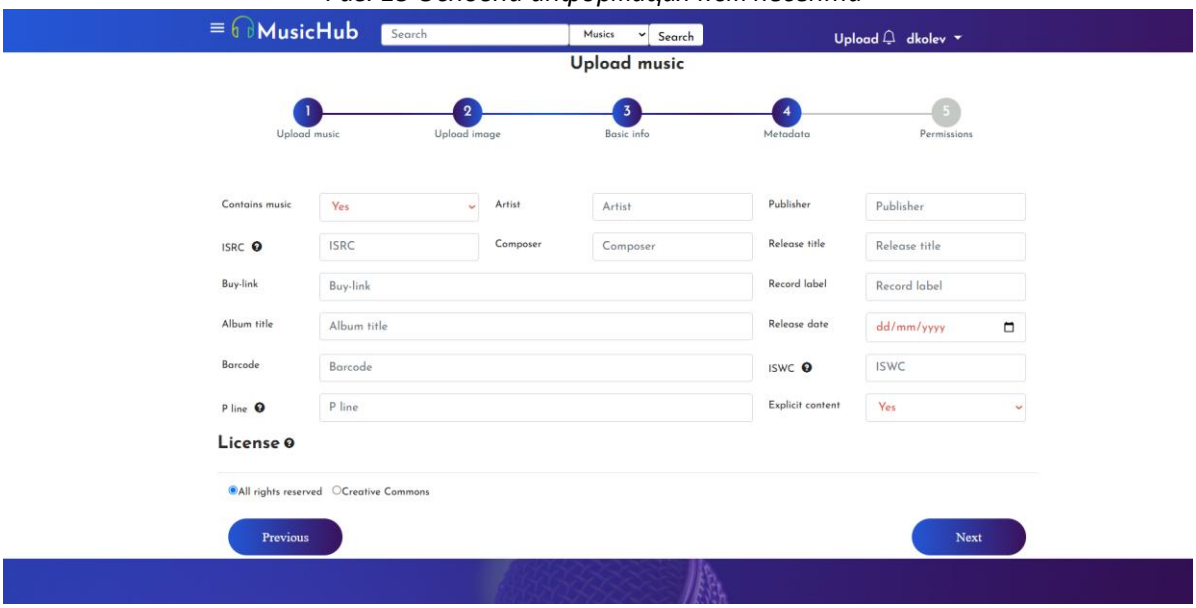
Фиг. 11 Добавяне на нова песен



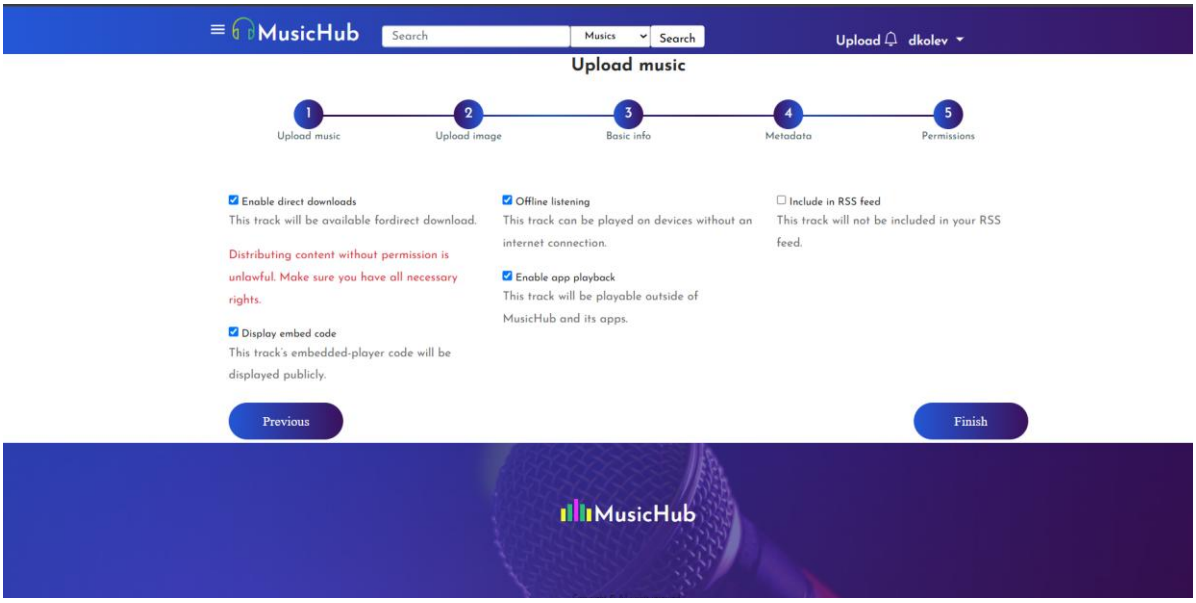
Фиг. 12 Добавяне на снимка към песента



Фиг. 13 Основна информация към песента



Фиг. 14 Metadata



Фиг. 15 Permissions

Първоначално потребителят качва музикалния файл, след което се добавя изображение, което служи като визуална корица на песента. В следващата стъпка се попълва основна информация, като заглавие, изпълнител и описание. След това се добавят допълнителни метаданни, които описват технически и съдържателни характеристики на файла. Последният етап включва конфигуриране на права за достъп, чрез които се определя кои потребители могат да слушат съответното музикално съдържание.

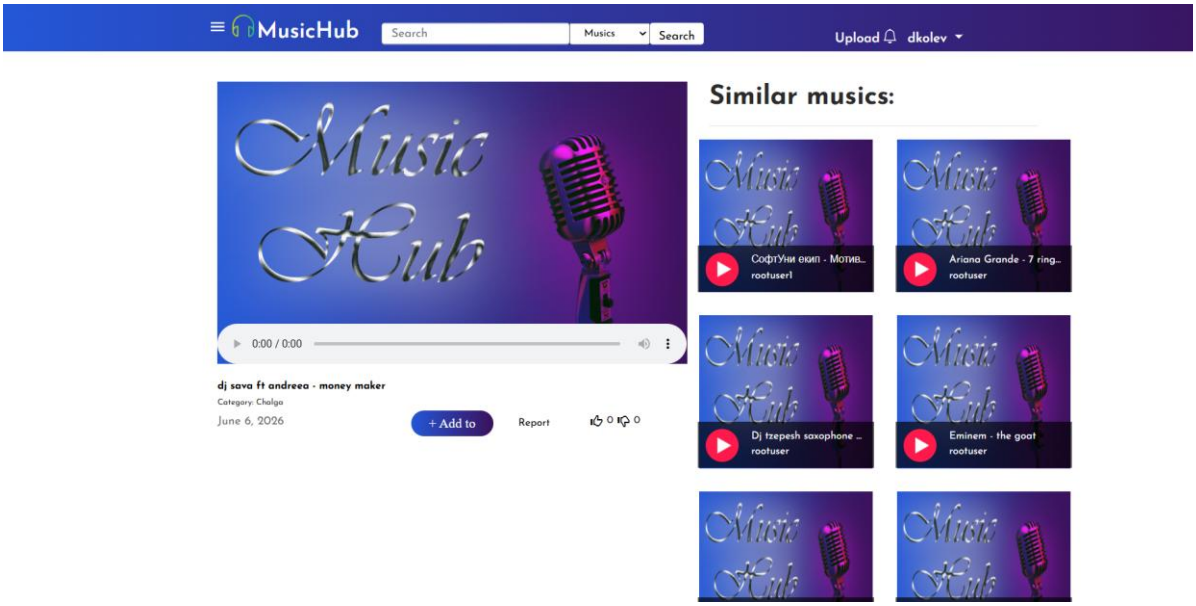
В текущата реализация файловете първоначално се съхраняват директно в директория на проекта, което не е оптимално решение от гледна точка на архитектура и скалируемост. Поради тази причина е предвидена бизнес логика за интеграция с външни storage услуги, като Azure Storage чрез AzureStorageService и Mega Storage чрез MegaStorageService. Тези услуги осигуряват по-надеждно и централизирано съхранение на файлове.

Важно е да се отбележи, че използването на тези външни storage услуги е свързано с разходи, тъй като те са платени решения. Поради това в рамките на проекта те се разглеждат единствено като архитектурно предвидена възможност, докато текущата имплементация е насочена към учебни и демонстрационни цели, а не към комерсиална експлоатация.

- **Слушане на песен**

Страницата за слушане на музика предоставя възможност на потребителя да възпроизвежда избрано музикално съдържание чрез вграден аудио плейър. Освен основната функционалност за слушане, платформата предлага и система за препоръка на подобни песни. Тези предложения се генерират на база историята на слушане на конкретния потребител и са изцяло базирани на предварително дефинирана бизнес логика, без използване на изкуствен интелект.

На същата страница потребителят разполага с допълнителни интерактивни функции, включително възможност за добавяне на песента към личен плейлист, харесване или нехаресване на съдържанието, както и подаване на сигнал (report) при необходимост. Тези действия позволяват персонализиране на потребителското изживяване и подпомагат управлението на съдържанието в платформата.

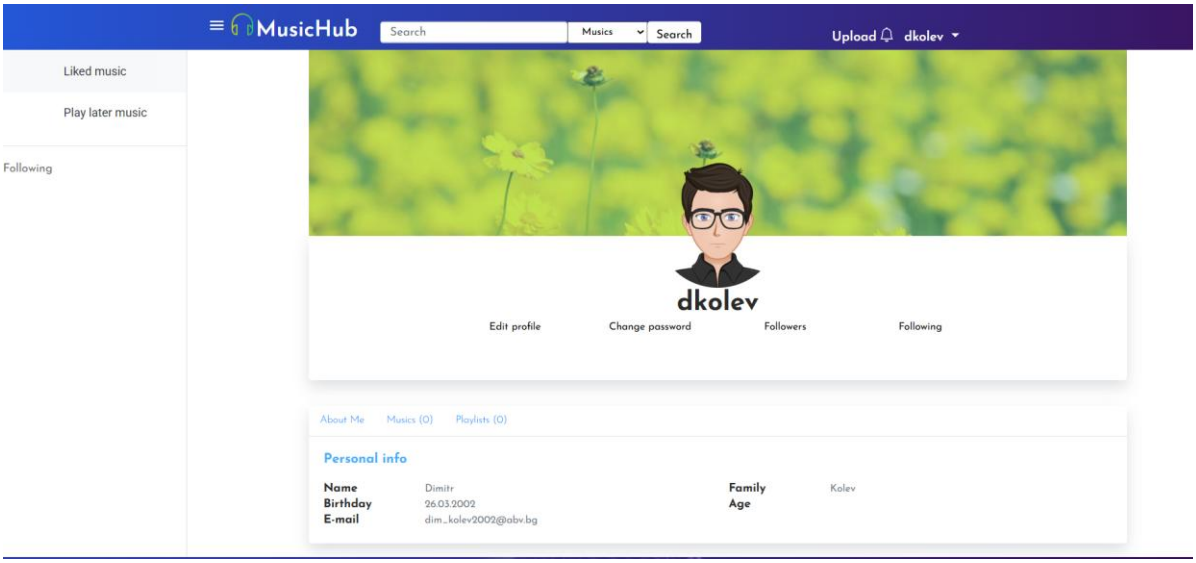


Фиг. 16 Страница за слушане на музика

- **Профил на потребителя**

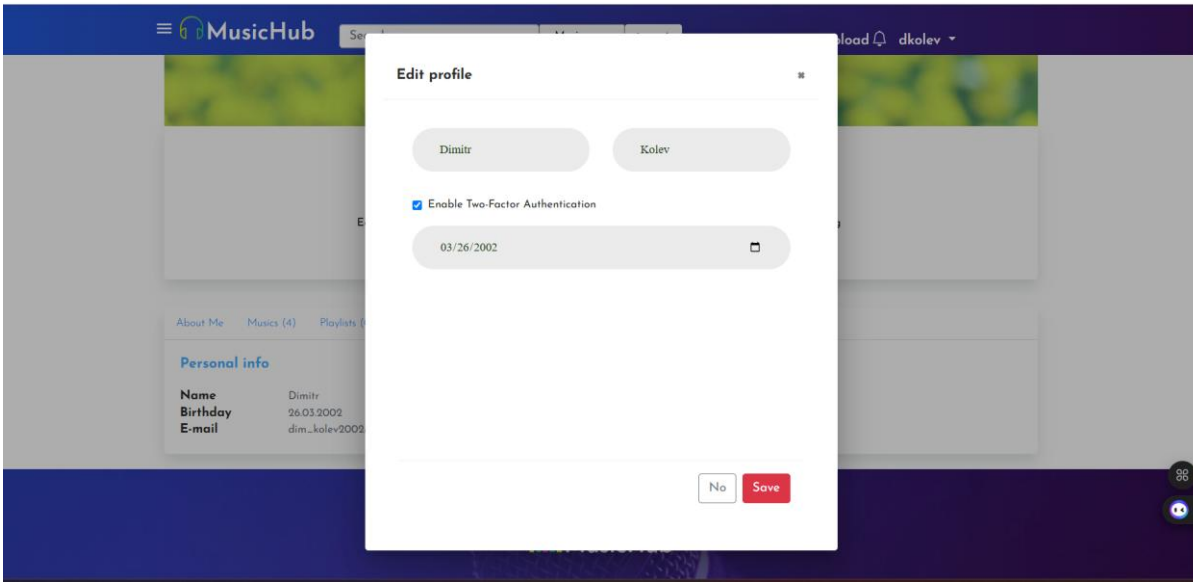
Страницата за потребителски профил предоставя възможност за управление и преглед на личната информация в системата. От тази секция потребителят може да редактира своите основни данни, както и да извърши промяна на паролата си с цел повишаване на сигурността на акаунта.

Освен настройки на профила, страницата включва и социални функционалности. Потребителят може да преглежда своите последователи, както и списък с профилите, които следва. По този начин се осигурява възможност за социално взаимодействие между потребителите в платформата и по-лесно проследяване на активността в мрежата.



Фиг. 17 Страница за слушане на музика

- **Изключване на двуфактурната автентикация**
  - Двуфакторната автентикация в системата може да бъде управлявана от потребителя чрез страницата за редакция на профил (Edit Profile). В появяващия се прозорец за редакция е налична опция под формата на отметка (checkbox) с надпис **Enable Two-Factor Authentication**.
  - Чрез активиране или деактивиране на тази опция потребителят има възможност да включва или изключва двуфакторната автентикация според своите предпочитания. При активирана настройка, при всяко влизане в системата се изисква допълнителна верификация чрез еднократен код, изпратен на електронната поща на потребителя.



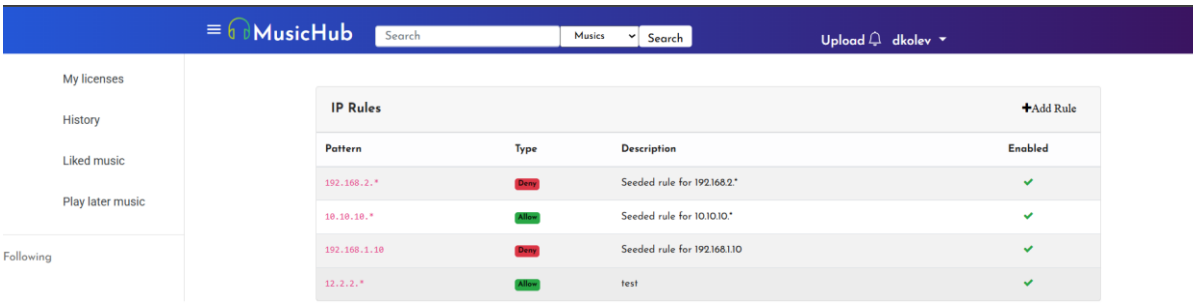
Фиг. 18 Редактиране на профил

- **Управление на IP конфигурация**

Страницата за IP конфигурация предоставя възможност за ограничаване на достъпа до профила или системата въз основа на IP адреси. Чрез нея потребителят или

администраторът може да дефинира списък с разрешени IP адреси, от които е позволено влизане в приложението, както и списък със забранени IP адреси, от които достъпът се блокира.

Тази функционалност се използва като допълнителен слой за сигурност, като позволява по-прецизен контрол върху мрежовия достъп до системата. По този начин се намалява рискът от неоторизиран достъп и се повишава общата защита на платформата.



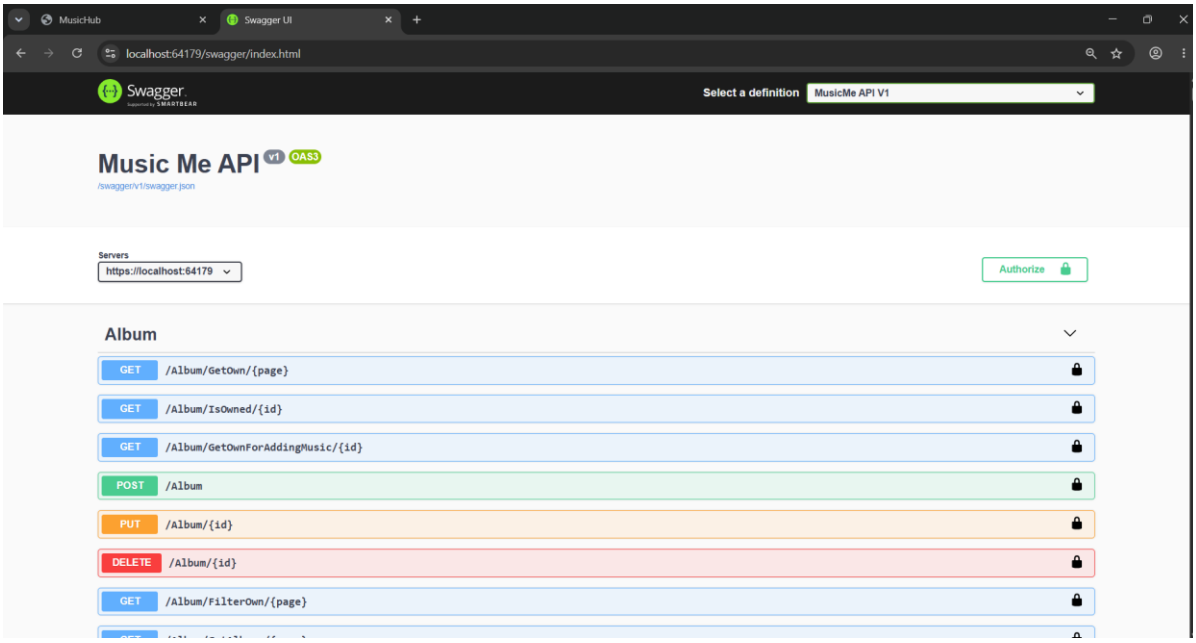
Фиг. 19 Страница Управление на IP конфигурация

- **Стартиране на swagger**

За тестване и документация на Web API в проекта е използван Swagger (OpenAPI). След стартиране на приложението Swagger интерфейсът е достъпен на адрес от вида:

https://localhost:64179/swagger/index.html

или на съответния порт, на който е стартирано приложението, последван от /swagger/index.html.

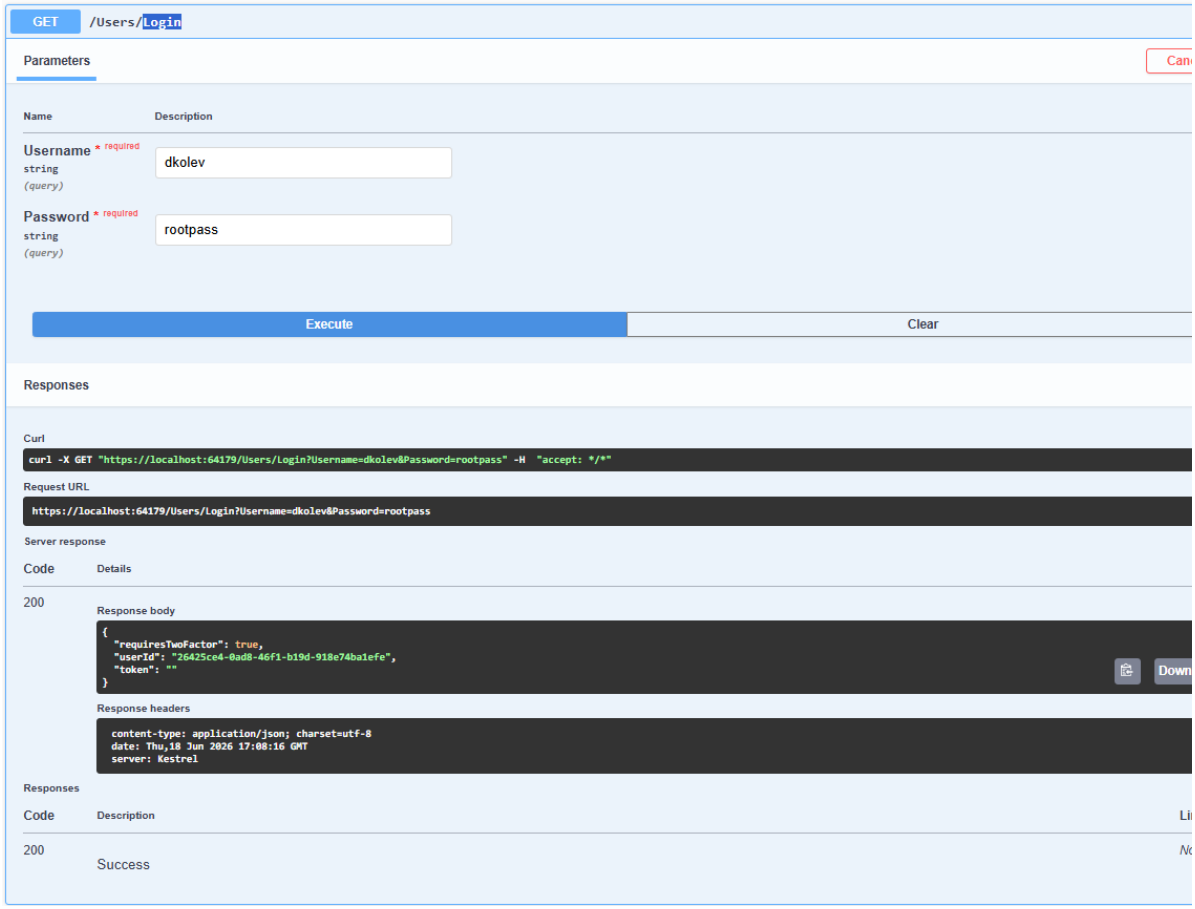


Фиг. 20 Swagger

В интерфейса на Swagger са визуализирани всички налични контролери и техните действия (Actions), като всяка операция може да бъде изпълнена директно през браузъра.

За удостоверяване на потребител първоначално се извиква методът POST /Users/Login, като се попълват потребителско име и парола. При успешно изпълнение методът връща JWT токен или изисква двуфакторна автентикация чрез електронна поща. В случай че е активирана двуфакторна автентикация, отговорът е подобен на следния:

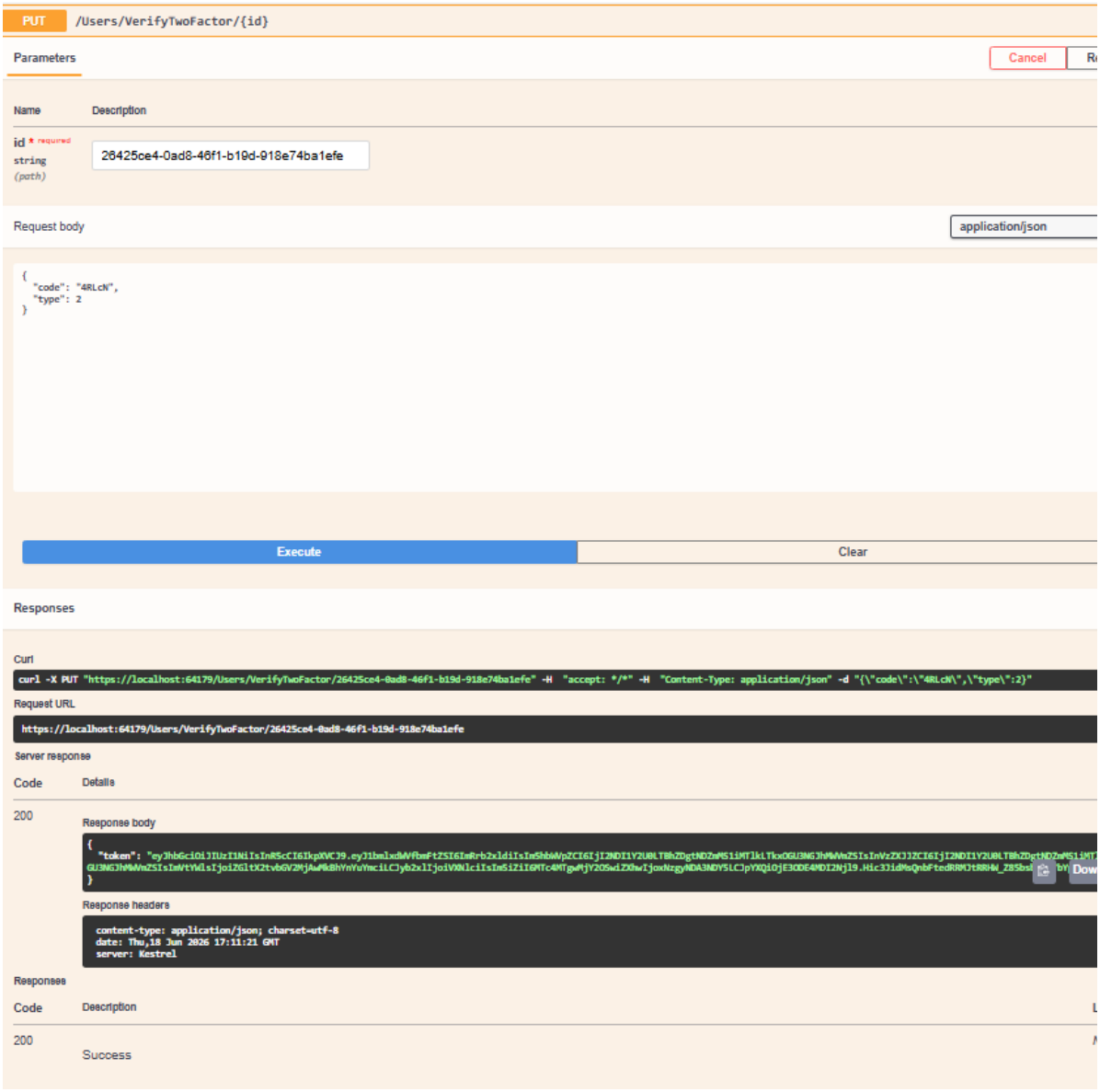
```
{
  "requiresTwoFactor": true,
  "userId": "26425ce4-0ad8-46f1-b19d-918e74ba1efe",
  "token": ""
}
```



Фиг. 20 Users/Login отговор

Когато е необходима двуфакторна автентикация, трябва да бъде извикан методът POST /Users/VerifyTwoFactor/{id}, където параметърът id представлява идентификатора на потребителя (userId), получен от предходната заявка. В тялото на заявката се изпращат полученият код за потвърждение и типът на двуфакторната автентикация:

```
{
  "code": "code",
  "type": 2
}
```

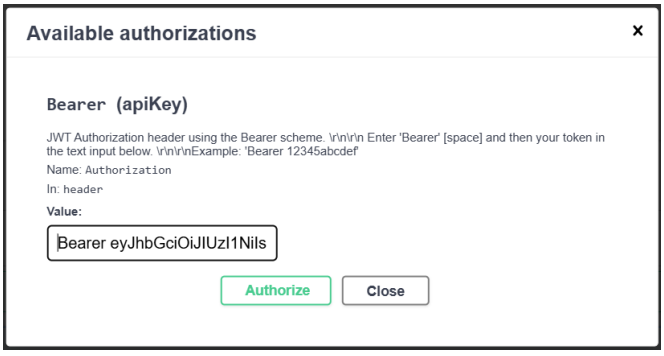


Фиг. 21 /Users/VerifyTwoFactor/{id} отговор

След успешно потвърждение методът връща JWT токен, който се използва за достъп до защитените крайни точки на приложението.

За използване на получения токен в Swagger се избира бутонът Authorize, след което в отворения прозорец се въвежда стойността в следния формат:

Bearer <JWT_TOKEN>



Фиг. 22 Логване чрез bearer token

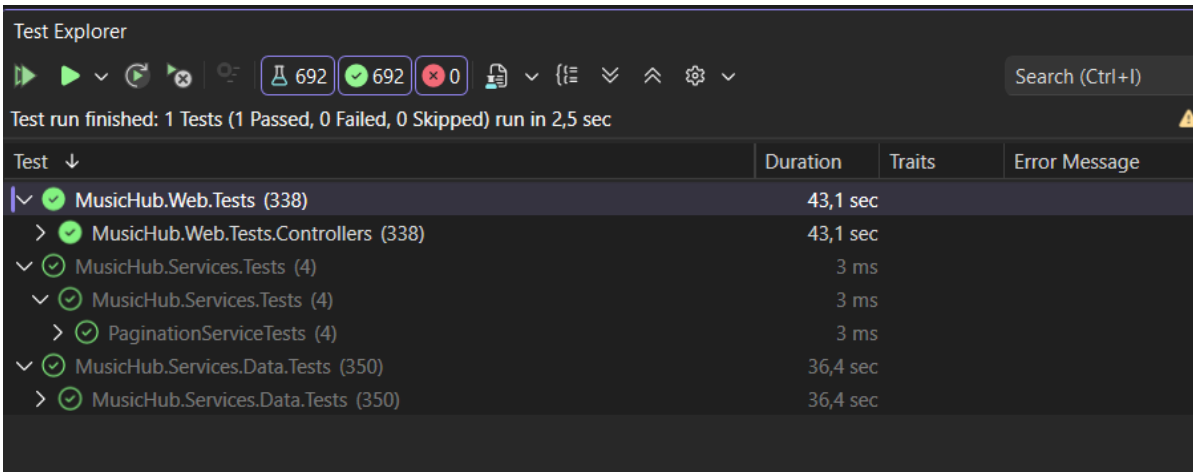
След успешна авторизация Swagger автоматично добавя токена към всички последващи заявки към защитените API методи, което позволява тяхното тестване без необходимост от повторно удостоверяване.

Стартиране на unit тестове

В разработената система са имплементирани unit тестове, които се използват за проверка на коректността на основни бизнес логики и функционалности. Първоначално проектът съдържа общо 692 теста, но поради необходимост от оптимизация и фокус върху критичните компоненти, голяма част от тях са премахнати, като са оставени 15 основни теста. Премахването е извършено с цел опростяване на тестовата среда и избягване на дублиращи се или нестабилни тестови сценарии.

Unit тестовете могат да бъдат изпълнени чрез вградения инструмент **Test Explorer** в Visual Studio. За целта потребителят трябва да отвори менюто **Test → Test Explorer**, след което в прозореца се визуализират всички налични тестове в проекта. Стартирането на тестовете се извършва чрез бутон **Run All** или чрез индивидуално изпълнение на избрани тестови случаи.

След изпълнение на тестовете Visual Studio предоставя подробен отчет за резултатите, включително успешно преминали тестове, неуспешни тестове и информация за възникнали грешки, което улеснява анализа и поддръжката на системата.



Фиг. 23 Резултати от Unit тестовете